JAVA COLLECTION AND FRAMEWORK PROGRAM

Name: Dhanusha D **RegisterNo:192424189**

Course: Data Handling and Visualization **Code: DSA0604**

Date:24-07-2026 **SLOT: D**

1

LOGGER

You are working on a project that requires the use of a logging framework. Write a Java program to create an interface called "Logger" with methods for logging different types of messages, such as "debug", "info", "warning", and "error". Implement the interface in a class called "Log4jLogger" that uses the Log4j logging framework.

CODE:

```
interface Logger
{
    void debug(String message);
    void info(String message);
    void warning(String message);
    void error(String message);
}
public class Log4jLogger implements Logger
{
    public void debug(String message)
    {
        System.out.println("[DEBUG] " + message);
    }
    public void info(String message)
    {
        System.out.println("[INFO] " + message);
    }
    public void warning(String message)
    {
        System.out.println("[WARNING] " + message);
```

```
    }
    public void error(String message)
    {
        System.out.println("[ERROR] " + message);
    }
    public static void main(String[] args)
    {
        Log4jLogger logger = new Log4jLogger();
        logger.debug("Debug Message");
        logger.info("Information Message");
        logger.warning("Warning Message");
        logger.error("Error Message");
    }
}
```

OUTPUT

```
[DEBUG] Debug Message
[INFO] Information Message
[WARNING] Warning Message
[ERROR] Error Message
```

2

CHARACTER

You are developing a game that involves different types of characters, such as heroes, villains, and monsters. Write a Java program to create an interface called "Character" with methods for moving, attacking, and taking damage. Implement the interface in classes for each type of character, such as "Hero", "Villain", and "Monster".

CODE:

```
interface Character
{
    void move();
    void attack();
    void takeDamage();
}
public class Hero implements Character
{
    public void move()
    {
        System.out.println("Hero is moving");
    }
    public void attack()
    {
        System.out.println("Hero attacks with a sword");
    }
    public void takeDamage()
    {
        System.out.println("Hero takes damage");
    }
    public static void main(String[] args)
    {
        Hero hero = new Hero();
        hero.move();
        hero.attack();
        hero.takeDamage();
    }
}
```

OUTPUT

```
Hero is moving
Hero attacks with a sword
Hero takes damage
```

3

SORT

Write a Java program to create a list of integers and sort them in ascending order using the Collections framework.

CODE:

```java
import java.util.*;
public class SortList
{
    public static void main(String[] args)
    {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(50);
        list.add(20);
        list.add(10);
        list.add(40);
        list.add(30);
        System.out.println("Before Sorting: " + list);
        Collections.sort(list);
        System.out.println("After Sorting: " + list);
    }
}
```

OUTPUT

```
Before Sorting: [50, 20, 10, 40, 30]
After Sorting: [10, 20, 30, 40, 50]
```

4

HASHSET

Write a Java program to create a HashSet of strings and remove a specific element from it.

CODE:

```java
import java.util.*;
public class HashSetExample
{
    public static void main(String[] args)
    {
        HashSet<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
        set.add("Orange");
        System.out.println("Before: " + set);
        HashSet<String> newSet = new HashSet<>();
        for (String s : set)
        {
            if (!s.equals("Mango"))
            {
                newSet.add(s);
            }
        }
        System.out.println("After: " + newSet);
    }
```

OUTPUT

```
Before: [Apple, Mango, Orange, Banana]
After: [Apple, Orange, Banana]
```

5

TREEMAP

Write a Java program to create a TreeMap of strings and integers and iterate over its entries.

CODE:

```java
import java.util.TreeMap;
import java.util.Map;
public class TreeMapExample
{
    public static void main(String[] args)
    {
        TreeMap<String, Integer> map = new TreeMap<>();
        map.put("Apple", 10);
        map.put("Banana", 20);
        map.put("Mango", 30);
        map.put("Orange", 40);
        System.out.println("TreeMap Elements:");
        for (Map.Entry<String, Integer> entry : map.entrySet())
        {
            System.out.println(entry.getKey() + " = " + entry.getValue());
        }
    }
}
```

OUTPUT

```
TreeMap Elements:
Apple = 10
Banana = 20
Mango = 30
Orange = 40
```

6

PRIORITYQUEUE

Write a Java program to create a PriorityQueue of integers and add elements to it.

CODE:

```java
import java.util.PriorityQueue;
public class PriorityQueueExample
{
    public static void main(String[] args)
    {
        PriorityQueue<Integer> queue = new PriorityQueue<>();
        queue.add(50);
        queue.add(20);
        queue.add(10);
        queue.add(40);
        queue.add(30);
        System.out.println("Priority Queue Elements: " + queue);
    }
}
```

OUTPUT

```
Priority Queue Elements: [10, 30, 20, 50, 40]
```

7

LINKEDHASHMAP

Write a Java program to create a LinkedHashMap of strings and integers and retrieve the keys in the order they were inserted.

CODE:

```java
import java.util.LinkedHashMap;
public class LinkedHashMapExample
{
    public static void main(String[] args)
    {
        LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
        map.put("Apple", 10);
        map.put("Banana", 20);
        map.put("Mango", 30);
        map.put("Orange", 40);
        System.out.println("Keys in insertion order:");
        for (String key : map.keySet())
        {
            System.out.println(key);
        }
    }
}
```

OUTPUT

```
Keys in insertion order:
Apple
Banana
Mango
Orange
```

8

MAX-MIN

Write a Java program to create a Set of integers and perform different operations on it, such as finding the maximum and minimum values.

CODE:

```java
import java.util.*;
public class SetExample
{
    public static void main(String[] args)
    {
        Set<Integer> set = new HashSet<>();
        set.add(25);
        set.add(10);
        set.add(40);
        set.add(15);
        set.add(30);
        System.out.println("Set: " + set);
        int max = Collections.max(set);
        int min = Collections.min(set);
        System.out.println("Maximum value: " + max);
        System.out.println("Minimum value: " + min);
    }
}
```

OUTPUT

```
Set: [40, 25, 10, 30, 15]
Maximum value: 40
Minimum value: 10
```